

Diffusion-MoE Phase 2 Training Investigation

Findings report — all-layers training run, bugs found, and current state

Prepared for review · diffusion-moe project

Executive Summary

Following Phase 1's diagnostics (`phase1_findings_report.md`) and the bounded single-layer pilot fine-tune, this phase asked the question those tools structurally couldn't answer: **what actually happens if every layer of the project's own from-scratch architecture is a `DiffusionMoELayer` , trained end-to-end, not just diagnosed or spliced into a frozen pretrained backbone?**

The honest answer is: six real, previously-undiscovered bugs, one badly-miscalibrated hyperparameter, and — after fixing all of them — clean, complete training runs that demonstrate the architecture can learn, route without permanently collapsing, and finish without crashing. That is real progress, not a finished result: a same-budget dense-model control, repeated across two seeds, decisively and consistently outperformed the all-layers diffusion-MoE configuration on quality, cost, *and* reliability (§6.4–6.5). See §7 for the full picture and what would change that conclusion.

1. **NaN divergence at step ~199** — `NystromDiffusionMap` 's kernel bandwidth (`eps_`) is fit once and frozen between landmark refits; once training moves the embedding distribution far enough, a token can drift outside every landmark's kernel support, and the unguarded division silently produces NaN, which `clip_grad_norm_` cannot repair. Fixed with both a defensive guard and — the deeper fix — a live `eps_` refresh between refits (§2.1).
2. **Checkpoint resume crashes a real MoE layer** — `NystromDiffusionMap` 's fitted state lives on a plain Python object, not an `nn.Module` , so it silently does not survive `state_dict()` save/load. Any resume off the exact refit-schedule boundary crashed outright. Found running the project's own `expert_attribution.py` tool against a real checkpoint, not by inspection (§2.2).
3. **No shared/always-active expert** — every FFN computation was contingent on sparse top-k routing; nothing had guaranteed, routing-independent gradient signal. Added a mandatory `shared_expert` , architecturally identical to a routed `ExpertFFN` , applied unconditionally outside the router (§3).
4. **Disk-full crash near the finish line** — unbounded checkpoint retention filled a 60GB instance's disk with ~5.7GB files, corrupting the final checkpoint write. Fixed with automatic pruning (§4).
5. **A catastrophic initialization bug** — `task_loss` started at ~823 nats against a random-guessing baseline of $\ln(32000)=10.37$, and never recovered below ~60 nats across any run, no matter how long training continued. Root-caused to two compounding, previously-unaddressed initialization gaps (no custom init existed anywhere in the model) and fixed; confirmed both in isolation and in a real training run (§5). This is the most consequential finding in this phase — everything observed in earlier runs (§6) was training on top of this bug.
6. **`mu_load` (the load-balance loss weight) was miscalibrated by ~2 orders of magnitude** relative to `task_loss` 's gradient once the loss scale was actually correct — measured directly, not guessed, and retuned (§6.2).
7. **`cfg.seed` was a no-op** — never applied to torch's global RNG anywhere in this pipeline (only `data.seed` , controlling shuffle order, was wired through), so model weight initialization — the dominant source of run-to-run variation — drew from whatever unseeded state the process happened to start in. Fixed, and what made the seed-repeat check in §6.5 meaningful in the first place (§8).

The final MoE run (`graceful-universe-21`) completed its full 2,441-step / 20M-token budget cleanly: zero NaN, `task_loss` ending at 7.08 (below the random baseline), all 24 layers' `load_loss` ending near zero (down from 7 of 24 near-total collapse mid-run), and `val_pp1` finally moving off its `exp(20)` safety cap for the first time in this investigation (7,286 → 5,081 → 35,294 spike → 16,158 → 3,245 → 2,239 → 974). That is real progress relative to every earlier run in this phase — but a same-budget **dense-model control run** (`peachy-cloud-22` , \$6.4), run immediately afterward, decisively beat it: `task_loss=4.80` , `val_pp1=93.2` , in ~15x less wall-clock time. A second seed of both configs (§6.5) confirmed this wasn't one lucky/unlucky comparison — dense stayed essentially unchanged (sub-1% variance), while the MoE run got worse on the second seed (`val_pp1=1,478` , wall-clock 425 min) rather than better. Read together, this phase's honest bottom line is: the infrastructure now works, and the current all-layers diffusion-MoE configuration is decisively and consistently worse than the simple baseline it needs to beat. See §7.

1. Background and Motivation

Phase 1's diagnostics and the bounded pilot fine-tune (`phase1_findings_report.md` §3.4–3.6) each tested one layer at a time, spliced into an otherwise-frozen pretrained Mistral-7B backbone. That structurally cannot answer a sharper question raised during review: *if a layer's router keeps forcing collapse onto one expert during real training, is that proof the layer can't benefit from diffusion routing at all, or just proof that one frozen-backbone, one-layer splice hasn't given the router anything to differentiate?* The only way to find out is to train the project's own from-scratch architecture with every layer routed, and watch. The user's explicit direction: *"I would let all layers use diffusion MoE and see what happens."*

This phase covers that run, the infrastructure it required (per-layer loss breakdown, checkpoint resume, expert-attribution analytics — see `methodology.md` §5 and §6 for the durable reference material), the bugs it surfaced, and the final result.

All runs in this section use the project's own 300M/24-layer from-scratch architecture (`configs/model/300m.yaml`), trained on `wikitext-103-raw-v1` via local parquet files (bypassing the recurring flaky-CDN Hub-streaming stall — see `methodology.md` §4), on a single rented RTX 3090 instance (vast.ai).

2. Bug: NaN Divergence, and the Deeper Fix Behind It

2.1 Root cause: a frozen kernel bandwidth under a moving distribution

The first all-layers run diverged to NaN at step 199, confirmed via an explicit `RuntimeWarning: invalid value encountered in divide` in `geometry/nystrom.py` . `NystromDiffusionMap.transform()` 's Nyström extension formula divides by each new point's total kernel mass to the (frozen) landmarks:

```
K_alpha = K_new / (d_alpha_new[:, None] * self._d_alpha_landmarks[None, :])
```

`eps_` (the kernel bandwidth) is derived once from the landmarks' own spread at the last refit (`fit()`), only run every `centroid_refresh_steps`) and then frozen until the next one. If a token's embedding drifts far enough from that frozen reference — plausible once `centroid_separation_loss` has been actively pushing centroids apart for hundreds of steps — its kernel value to every landmark can underflow to exactly `0.0` in float64 (`exp(v)` underflows to the literal bit pattern `0.0` below roughly `v < -745` , not a small-but-nonzero value). The resulting `0/0` division produced NaN , which propagated through `task_loss` / `load_loss` , and — critically — `torch.nn.utils.clip_grad_norm_` cannot repair a NaN gradient (the norm of a NaN tensor is itself NaN), so every parameter update after that point silently corrupted the entire model. Training continued to "run" for ~40 more steps on fully poisoned weights before

crashing outright on an unrelated downstream `KMeans` call (sklearn's input validator rejects NaN explicitly; the raw numpy division earlier did not).

Immediate fix: guard the division (`d_alpha_new_safe` , mirroring the existing `row_sums` guard a few lines below) so a token with zero kernel mass gets diffusion coordinates at the origin (maximal uncertainty) instead of NaN.

2.2 The real fix: live-refreshing the kernel bandwidth between refits

The guard alone treats a symptom. The actual staleness problem — `eps_` describing a step-0 snapshot of the embedding distribution, growing more wrong the longer training runs between refits — remains. Fixed by having `transform()` re-derive `eps_` from the *current* batch's actual distance to the (still-frozen) landmarks on every call, reusing the distance matrix already computed for the Nyström kernel (cheap — no new k-means).

This surfaced two further correctness requirements, both needed for the fix to be mathematically sound rather than merely avoid a crash:

- **eigenvectors_ / psi_landmarks_ must be re-derived alongside `eps_`** (extracted into a shared `_fit_landmark_spectrum` , called by both `fit()` and the refresh path): Nyström's defining correctness property — that applying the extension formula to the landmarks themselves must exactly reproduce their direct diffusion coordinates — only holds when the landmark spectrum and the new-point extension share one consistent kernel. Caught by `test_nystrom_quality_exact_reproduction_on_landmarks` when a first attempt (refreshing only `eps_` in isolation) broke it (0.0042 error against a $1e-4$ tolerance).
- **diffusion_eigenvectors 's ARPACK solve needed a seeded starting vector.** It previously ran once per landmark set, so its default unseeded random start never mattered. Once it could rerun every step, an unseeded start would make the diffusion-coordinate basis jitter step-to-step from solver randomness alone — independent of any real drift in the data, i.e. the opposite of what this fix was for.
- **fit_transform 's own internal `transform()` call must skip the refresh** (new `_refresh_eps` parameter): it runs immediately after `fit()` on the same data, so there is no staleness yet to correct, and refreshing anyway would silently override `fit()` 's landmark-based heuristic and redo the eigensolve twice on every refit step for no benefit.

Cost: this is not free, but it is cheap relative to what it replaces. `eps_` 's refresh reuses an already-computed distance matrix; the re-derived eigensolve is over just the `n_landmarks × n_landmarks` matrix (32–128 points), not another k-means clustering over the full token batch. Measured directly in production: steady-state per-step time rose from ~5.55s to ~7.0–7.3s (roughly 25–30%), on top of an already CPU-bound geometry pipeline (this entire subsystem runs on `z.detach()` 'd numpy/sklearn, never touching the GPU).

3. Feature: A Mandatory Shared Expert, Outside the Router

Once every layer was routed and training could be observed cleanly, a striking pattern emerged (visible even before the initialization bug was found, §5): `load_loss` stayed elevated across nearly all 24 layers simultaneously — not one collapsed outlier, a near-universal, if partial, imbalance (mean `load_loss` 3.21 across all layers at one snapshot, only 1 of 24 layers below 1.0). This raised the mechanistic question motivating this whole run: with every FFN computation contingent on sparse top-k routing, nothing in the network had guaranteed, routing-independent gradient signal — common/generic computation had to be either redundantly relearned by whichever expert won that step's routing lottery, or not learned reliably at all if routing was noisy.

This is a known pattern in production MoE architectures (DeepSeekMoE's "shared expert isolation," similarly Qwen2-MoE), added here as `DiffusionMoELayer.shared_expert` : architecturally identical to a routed `ExpertFFN` (same width, same `overlap_factor`), applied unconditionally to every token, every step — no gate value, no centroid, entirely outside the diffusion router's influence. This adds real active compute per token (`top_k + 1` experts fire instead of `top_k`), not a reallocation of existing capacity.

Verified with a direct behavioral test

(`test_shared_expert_contributes_even_when_routing_contributes_nothing`): with the routed dispatch path monkeypatched to contribute exactly zero, the layer's output still differs from the plain attention residual — the shared expert's contribution cannot be made routing-contingent by construction.

4. Bug: Disk-Full Crash Near the Finish Line

A run that had otherwise reached step 2,400 of 2,441 (98.3%) crashed on its own checkpoint save: the instance's 60GB disk filled completely (8 accumulated checkpoints × ~5.7GB each), and the final write was cut off mid-file (`PytorchStreamWriter failed writing file` ; the resulting `step_2400.pt` was truncated to ~2.97GB against an expected ~5.68GB — corrupt, unusable). The last *valid* checkpoint (`step_2100.pt` , saved before the disk filled) was confirmed to load cleanly.

Fixed with automatic checkpoint retention in `Trainer` : after each numbered checkpoint save, all but the most recent `keep_last_n_checkpoints` (default 2, configurable via `training.keep_last_n_checkpoints`) are deleted. `best.pt` is a separate file and always kept regardless. `<= 0` disables pruning (old, unbounded behavior) for anyone who wants it.

5. The Initialization Bug: Task_Loss Was Never Actually Calibrated

5.1 The anomaly

Every run before this fix showed the same signature: `task_loss` starting around 823 nats (step 1) and never recovering below roughly 60 nats, however long training continued. `val_ppl ( exp(min(avg_nll, 20.0)) )` was pinned at its safety cap — 485,165,195.4 exactly — at *every single logged evaluation, across every run, all the way to the end of a 2,400-step run*. This is a real red flag independent of routing behavior: this project's tokenizer (Mistral's, 32,000 tokens) gives a random-guessing baseline of `ln(32000) ≈ 10.37` nats — the cross-entropy a properly-initialized, completely untrained model should sit near. A trained model that never gets within even a factor of 2 of that baseline, in either direction, is not "undertrained" — something is numerically broken.

5.2 Root cause, confirmed by direct measurement

No custom weight initialization existed anywhere in the model — every linear layer and the token embedding relied entirely on PyTorch's defaults. Two compounding gaps, confirmed empirically (not just inferred) by hooking every block of a freshly-constructed 24-layer model and measuring residual-stream norms and final logit magnitude directly:

1. `token_embedding` used `nn.Embedding` 's default init — `Normal(0, 1)` , `std=1.0` — far too large for a transformer embedding table, and especially costly because `tie_embeddings=True` by default makes this same oversized matrix double as the LM-head unembedding projection, directly inflating logit magnitude. This was the *dominant* cause: fixing only the second gap below (residual-output scaling) left final logits unchanged — actually slightly worse (`max abs` 828 → 1071) — until this was also fixed.

2. **No depth-aware scaling on residual-branch output projections** (attention's `out_proj`, every FFN's `down_proj` — dense `TransformerBlock`, routed `ExpertFFN`, and the new `shared_expert` alike). Every serious transformer implementation (GPT-2, LLaMA, nanoGPT) scales these down by $1/\sqrt{2 \times n_layers}$ at init specifically because residual-stream variance otherwise compounds with depth. Confirmed directly: residual-stream norm grew $32 \rightarrow 44$ over 24 layers unfixed; a controlled, smooth $1.0 \rightarrow 7.1$ fixed.

Combined fix: `token_embedding` initialized to the standard `std=0.02` (GPT-2/nanoGPT convention); every `out_proj` / `down_proj` weight scaled by $1/\sqrt{2 \times n_layers}$ after construction (matched on the Linear's own attribute name, so it applies uniformly regardless of which module — dense, routed expert, or shared expert — it's nested in).

Verified twice. In isolation: a fresh random-init model's cross-entropy against random labels dropped from $\sim 1,014$ to **10.51** (essentially exactly $\ln(32000)=10.37$). In real training (`vital-microwave-20`, the first run with this fix): step 1 `task_loss` = **10.61**, and a clean, sane trajectory afterward ($10.61 \rightarrow 9.03 \rightarrow 8.47$ by step 26) — nothing resembling any earlier run's trajectory.

5.3 What this means for everything observed before this fix

`resilient-brook-18` and `hardy-lake-19` (§6) both trained on top of this bug. Their qualitative conclusions (per-layer collapse patterns, shared-expert comparison) are likely still valid as *relative* signals within those runs, but their absolute loss numbers should not be treated as meaningful, and the shared-expert-vs-not comparison in §6.2 should be re-run now that the loss scale is correct before drawing any firm conclusion from it.

6. Run-by-Run Summary

Run (wandb)	Init fix	Shared expert	<code>mu_load</code>	Outcome
<code>resilient-brook-18</code>	No	No	0.01	<code>task_loss</code> plateaued 60–110 nats; <code>val_pp1</code> permanently capped; killed to add the shared expert
<code>hardy-lake-19</code>	No	Yes	0.01	Modest improvement over the above at matched steps (<code>task_loss</code> =90.3 vs 100.3 @ step 600) but still broken-scale; crashed on disk-full at step 2,400/2,441 (§4)
<code>vital-microwave-20</code>	Yes	Yes	0.01	Step 1 <code>task_loss</code> =10.61 (correct!), but 18 of 24 layers collapsed to <code>load_loss</code> ≈7.0 (the exact CV ² ceiling) by step ~600; <code>task_loss</code> climbed back to 11.5 — worse than its own starting point; killed for <code>mu_load</code> retuning
<code>graceful-universe-21</code>	Yes	Yes	2.0	Completed cleanly — see §6.3
<code>peachy-cloud-22</code> (dense control, <code>layers_to_replace=[]</code>)	Yes	n/a	n/a	Same budget, same init. Beat graceful-universe-21 decisively — see §6.4
<code>sunny-surf-23</code> (dense, <code>seed=123</code>)	Yes	n/a	n/a	Near-identical to <code>peachy-cloud-22</code> — see §6.5

cool-pyramid-24 (MoE, seed=123)	Yes	Yes	2.0	Completed cleanly, but meaningfully worse and slower than graceful-universe-21 — see §6.5
---------------------------------	-----	-----	-----	---

6.1 The collapse dynamic got *worse*, not better, once the loss was fixed

`vital-microwave-20` is the sharpest result in this phase. Fixing the initialization did not just fix the loss scale — it let gradients flow cleanly for the first time. `task_loss`'s gradient became a strong, well-calibrated signal instead of one dominated by first fixing a catastrophic starting point, which made the router-collapse shortcut (route everything to one expert per layer, reducing the effective optimization problem) *more* attractive to find, not less. 18 of 24 layers reached essentially the exact `load_loss` ceiling ($n_experts - 1 = 7.0$) within ~600 steps. This is a direct escalation of the question this whole run was designed to answer: collapse is not a rare pathology in one unlucky layer, it is close to the *default* outcome once the network can actually learn efficiently, unless something resists it hard enough.

6.2 Measuring, not guessing, the fix

Rather than guess a new `mu_load`, the relative gradient pull of `task_loss` vs. `load_loss` on the shared `ExpertCentroids` parameter was measured directly on a fresh, correctly-initialized model (the parameter both losses actually compete to move, since `load_loss`'s gradient is confined to centroids — `Psi_t` is detached before the routing-loss computation, so it never reaches attention/backbone weights directly). Raw ratio: `task_loss`'s pull was only **1.86×** `load_loss`'s — a fair fight. The old `mu_load=0.01` shrank that to an effective **186×**, leaving `load_loss` almost no real ability to resist collapse — a value very likely tuned against the old, catastrophically-mis-scaled `task_loss` and never revisited once §5's fix corrected it. Set to `mu_load=2.0`, targeting an effective ratio near 1.

6.3 The final run

`graceful-universe-21` completed the full 2,441-step budget cleanly — zero NaN, `state=finished`. At the halfway point (step ~296), the picture was a real but partial improvement: 7 of 24 layers near collapse, versus 18 of 24 in the unfixed-`mu` run at a comparable point — down ~60%, not eliminated. By the *end* of training, the picture resolved further: all 24 layers' `load_loss` near zero (best 0.008, most below 0.0005) — though the path there was genuinely volatile, not a clean monotonic decrease (`load_loss` oscillated between 0.45 and 3.6 across steps 1,200–2,200 before settling as the learning rate decayed toward its floor; see §7 for why this specific ending should not yet be over-interpreted).

`task_loss` ended at **7.08** — below the random baseline, genuine learning. `val_ppl` moved for the first time in this entire investigation, off the `exp(20)` cap:

```
step 300: 7,286
step 600: 5,081
step 900: 35,294  <- real spike, not a fluke
step 1200: 16,158
step 1800: 3,245
step 2100: 2,239
step 2400: 975   <- best of the run
```

Non-monotonic and still high in absolute terms for a 300M-parameter model (even on a small budget), but a real, measured signal that the model is generalizing at all — something no earlier run in this entire investigation had shown.

6.4 The dense-model control, and a decisive negative result

Immediately after `graceful-universe-21` completed, the same config was rerun with `layers_to_replace=[]` (dense baseline, identical data/budget/token count/hyperparameters otherwise) — the missing control flagged in every version of §7 until now. Same starting point confirmed (`task_loss=10.59` at step 1 vs. the MoE run's `10.61` , so the initialization fix applies uniformly and this is a fair comparison):

	Dense (peachy-cloud-22)	Diffusion-MoE, all layers (graceful-universe-21)
Final task_loss (step 2441)	4.80	7.08
Final val_ppl	93.2	974.6
val_ppl trajectory	Smooth, monotonic: 274 → 200 → 156 → 131 → 100 → 95 → 93	Noisy, huge spike at step 900: 7,286 → 5,081 → 35,294 → 16,158 → 3,245 → 2,239 → 975
Wall-clock, same 2,441 steps	20.1 min	294.5 min (~4.9 hours, ~15x longer)

The dense model wins decisively on every axis, at matched token budget. The time gap is almost entirely the CPU-bound Nyström/eigensolve pipeline (~7 s/step MoE vs. ~0.49 s/step dense — matches the ~15x wall-clock ratio directly), not GPU compute. This is not even a "worse quality, but cheaper active compute" tradeoff in MoE's favor: the diffusion-MoE model's active FFN width per token (`top_k+1=3` experts × 512 each = 1,536) is *smaller* than the dense model's full 4,096-wide FFN — less active capacity, ~15x the wall-clock cost, and a meaningfully worse result.

This is a real, decisive answer, and it is unfavorable to the current approach. Nothing in this investigation shows the added complexity of diffusion-routed MoE buying anything at this scale/budget. The standard framing for why sparse MoE is worth its complexity (Clark et al. 2022's scaling-law work, referenced earlier in this investigation) is specifically about quality-per-active-FLOP *at scale* — 20M tokens on a 300M-parameter model is nowhere near the regime that literature has actually demonstrated an advantage in. This result is consistent with that: it does not show diffusion-MoE is a bad idea in general, but it does mean this specific comparison offers no evidence *for* it, and meaningful evidence against training it further at this scale without a specific, articulated reason to expect the gap to close only at larger scale.

6.5 Seed-repeat check: dense is consistent, diffusion-MoE is not

Both §6.4 configs were rerun with `seed=123` (a new fix this phase: `cfg.seed` was previously never actually applied to torch's global RNG — only `data.seed` , controlling shuffle order, was wired through, meaning every prior run's model initialization was drawn from whatever unseeded state the process happened to start in, not a controlled value at all — see §8).

	seed=42	seed=123	Spread
Dense task_loss	4.80	4.80	~0%
Dense val_ppl	93.2	92.8	<1%
Dense wall-clock	20.1 min	20.0 min	~0%

MoE task_loss	7.08	7.44	~5%
MoE val_ppl	974.6	1,477.5	~52%
MoE wall-clock	294.5 min	424.6 min	~44%
MoE load_loss (final)	0.00044	0.00020	both ≈0

Two things confirmed here. First, §6.4's dense-beats-MoE result is not a fluke of one lucky/unlucky seed pair — dense is close to seed-invariant, and the *worse* of the two MoE seeds (seed=123, val_ppl=1,478) is still over 15x worse than *either* dense run. Second, and a genuinely new finding rather than just confirmation: diffusion-MoE is not just worse on average, it is meaningfully *less consistent* than dense, on both quality and cost — a 52% perplexity swing and a 44% wall-clock-time swing between two runs that differ only in random seed, versus dense's sub-1% variation on both. This extends Phase 1 §3.5's seed-42-vs-123 finding (different experts collapsing depending on seed) rather than just replicating it: it's not only *which* expert collapses that's seed-sensitive, it's the *total training cost and final quality* too — a real, separate cost of the current approach's variance, independent of its average performance gap.

6.6 What token-level routing actually looks like (early-training snapshot)

scripts/expert_attribution.py (extended this phase with Hydra CLI override support, so it can point at whatever data config a checkpoint was actually trained against — see methodology.md §6) was run against hardy-lake-19 's step-300 checkpoint. Filtering out padding (a first pass mistakenly included it — 479 of 492 positions in the sampled sequence were padding, trivially uniform) revealed real, reproducible structure, but **mostly positional, not semantic**, at this early stage: layer 0 routes only the first 1–4 tokens of every sequence to one expert (sequence-start detection) and everything else to another, regardless of topic; layer 6 shows a clean depth-progression through three experts as a sequence gets longer, independent of content; layer 20 was fully collapsed (one expert, ~99% of all real tokens across 5 sampled sequences). One layer (7) showed a plausible content-correlated split — a block of numeric/statistical text routing differently than surrounding narrative prose — but with one sequence and no position control, this could not be distinguished from coincidence. Worth repeating against a later, better-trained checkpoint (§7).

7. Current State and What Would Be Needed Before Scaling Up

What this phase established: the infrastructure-level failure modes are found and fixed — NaN divergence, checkpoint resume, disk exhaustion, and (the big one) the initialization bug that made every prior run's absolute numbers meaningless. One full run now completes cleanly end-to-end with sane losses throughout. Now that the missing control has been run (§6.4) *and* repeated with a second seed (§6.5): **at this scale and budget, the dense baseline beats the all-layers diffusion-MoE model decisively and consistently — better task_loss , an order-of-magnitude-plus better val_ppl on both seeds, and 15-21x less wall-clock time — while also being far more seed-consistent than the MoE configuration (sub-1% run-to-run variance vs. MoE's 44-52%)**.

Remaining open questions:

- 1. The final near-zero load_loss coincides with the learning rate being fully decayed, on both seeds.** It is plausible routing balance was actively, robustly resolving itself in both cases — and equally plausible it simply froze wherever it happened to be as gradients vanished toward the end of the cosine schedule, which would not be evidence of a stable equilibrium at all. Two data points now both showing the same end-of-run pattern makes the "real equilibrium" reading somewhat more credible than after §6.3 alone, but doesn't settle it.

2. **20M tokens is a smoke-test budget**, not a real pretraining scale, and the standard case for why sparse MoE is worth its complexity (Clark et al. 2022's routed-vs-dense scaling laws, discussed earlier in this investigation) is specifically about quality-per-active-FLOP *at scale* — this comparison is nowhere near the regime that literature has actually demonstrated an advantage in. §6.4/§6.5's results do not show diffusion-MoE is a bad idea in general; they show this specific comparison offers no evidence for it, twice.

Recommendation, revised again: not ready for a scaled-up run, and the seed-repeat result (§6.5) removes the most likely objection to that conclusion — this was not one unlucky comparison. Before spending more compute on the current diffusion-MoE approach at all: (a) profile and reduce the CPU-bound Nyström/eigensolve cost specifically — a 15-21x wall-clock penalty needs a very large quality win to ever justify itself, and right now the result runs the other direction; (b) revisit whether all-layers replacement was the right configuration to keep pursuing — Phase 1's original oracle-ceiling work recommended a *selective* few layers, not all 24, which this investigation deliberately set aside to answer "what happens if every layer routes." That question has now been answered, fairly conclusively, at this scale: worse and less consistent than not routing at all. The selective-layer approach remains untested by this phase and is the more promising next direction, not a bigger version of what was just run.

8. Code Changes Summary

File	Change
geometry/nystrom.py	Guarded the <code>d_alpha_new</code> division against exact-zero underflow; added live <code>eps_/landmark-spectrum</code> refresh in <code>transform()</code> (new <code>_refresh_eps</code> param, default on); extracted <code>_fit_landmark_spectrum</code> shared between <code>fit()</code> and the refresh path
geometry/eigensolver.py	<code>diffusion_eigenvectors</code> gained a <code>random_state</code> param, seeding ARPACK's starting vector — needed once the eigensolve could rerun mid-training, not just once per landmark set
models/moe_layer.py	Added mandatory <code>shared_expert</code> (unconditional, outside the router); <code>_compute_diffusion_coords</code> now also refits when <code>ndm.landmarks_</code> is <code>None</code> (checkpoint-resume fix), not only on the step-count schedule
models/moe_model.py	Fixed <code>token_embedding</code> init (<code>std=0.02</code> , was <code>nn.Embedding</code> default <code>std=1.0</code>); added depth-aware <code>1/sqrt(2×n_layers)</code> scaling to every <code>out_proj/down_proj</code> weight after construction
training/trainer.py	Added non-finite-loss guard in <code>train_step</code> (raises <code>FloatingPointError</code> before <code>backward()/optimizer.step()</code> instead of silently training on <code>NaN/Inf</code>); added <code>_prune_old_checkpoints</code> (<code>keep_last_n_checkpoints</code> , default 2)
data/dataloader.py	Fixed <code>local_data_files</code> reaching the real Hydra-config-driven training path — a <code>ConfigAttributeError</code> several frames deep (<code>omegaconf.ListConfig</code> vs. plain <code>list</code>), not an obvious error at the call site
scripts/expert_attribution.py	Added Hydra-style CLI override support (<code>parse_known_args</code>), so it can point at the real data config a checkpoint was trained against instead of always falling back to <code>base_config.yaml</code> 's defaults
scripts/train.py	<code>cfg.seed</code> now actually applied (<code>torch.manual_seed</code> / <code>np.random.seed</code> / <code>random.seed</code> / <code>torch.cuda.manual_seed_all</code>)

	before model construction — previously a no-op
configs/base_config.yaml	routing.mu_load: 0.01 → 2.0 (measured, §6.2); training.keep_last_n_checkpoints: 2 (new)

All changes are covered by tests reproducing the specific failure before confirming the fix, not just testing the fix in isolation — e.g. a mixed-batch outlier reproducing the exact NaN-underflow condition; a cross-object checkpoint resume at a step deliberately off the refit schedule, confirmed to crash on the pre-fix code (verified by temporarily reverting the fix) and pass after; a monkeypatched zero-contribution routed path confirming the shared expert's contribution is structurally unconditional; a real disk-bounded checkpoint-pruning scenario; the random-label cross-entropy check confirming init-time loss lands near the true random baseline (confirmed to fail — 61.5 vs. an expected <13.8 — on the pre-fix code); and a same-seed-reproduces/different-seed-diverges check on model initialization. Full suite: 376 tests passing at last check.

Appendix: Open Questions Carried Forward

- Does the depth-progression / sequence-start routing pattern (§6.6) persist, sharpen into content-sensitivity, or dissolve entirely at a later training step and a larger token budget?
- Should `nu_sep` (currently unchanged at 0.05) be measured the same way `mu_load` was (§6.2)? `sep_loss` showed real instability of its own (swings from -3.5 to -10.6 within ~120 steps in the collapsed run) that was not directly investigated — it may be a downstream symptom of the same collapse dynamic, or a separate miscalibration.
- The self-tuning (per-point adaptive) bandwidth design discussed but not implemented (the "Option 2" alternative to §2.2's global live-refresh) remains a candidate if staleness-driven instability recurs even with the current fix.
- `NystromDiffusionMap`'s own k-means/eigensolve randomness still has a hardcoded `random_state=42`, independent of `cfg.seed` (§8) — the seed-repeat check in §6.5 varied model init and data-adjacent RNG state, but not this. Worth wiring through too if the seed-sensitivity finding in §6.5 needs isolating further (is it driven by init, by the diffusion-map's own landmark selection, or both).